9.1 HTTP 1 vs HTTP 2

Эволюция протокола и практика HTTP-сервера на Go.

- Сообщения, методы и коды ответа.
- Повторное использование соединения и мультиплексирование.
- Маршруты, параметры и JSON в net/http.

HTTP: запрос и ответ

- Клиент обращается к ресурсу; сервер возвращает статус, заголовки и при необходимости тело.
- Метод описывает намерение клиента. URL задаёт адрес ресурса и параметры.
- HTTP/1.x и HTTP/2 сохраняют основную семантику методов и статусов; различается передача сообщений.
- Браузер, curl и Postman позволяют отправить запрос и изучить ответ.

Развитие протокола

- Ранний HTTP был ориентирован на получение документов.
- HTTP/1.0 расширил формат сообщениями с заголовками, методами и статусами.
- HTTP/1.1 добавил постоянные соединения по умолчанию, требования к Host, pipelining и chunked-передачу.
- HTTP/2 использует бинарные кадры и потоки, сохраняя смысл HTTP-запросов.

Как выглядит HTTP/1.1

Запрос и ответ показаны последовательно; окончания строк в протоколе CRLF

```
GET /hello?name=Go HTTP/1.1
Host: localhost:8080
Accept: text/plain

HTTP/1.1 200 OK
Content-Type: text/plain; charset=utf-8
Content-Length: 9

Hello Go!
```

- Пустая строка отделяет заголовки от тела.
- Host указывает целевой узел. Content-Type описывает формат данных.

Постоянное соединение и pipelining

- HTTP/1.1 обычно повторно использует TCP-соединение; отдельный Keep-Alive для включения этого режима не требуется.
- Pipelining позволяет отправить запросы, не ожидая каждого ответа.
- На одном таком соединении ответы должны идти в порядке запросов: медленный ответ задерживает последующие.
- Параллельные соединения могут уменьшать ожидание, но увеличивают число соединений.

Что меняет HTTP/2

- Сообщения разбиваются на бинарные кадры, относящиеся к отдельным потокам.
- Кадры разных потоков можно чередовать внутри одного TCP-соединения.
- Заголовки кодируются с помощью HPACK, что сокращает повторяющиеся метаданные.
- Данные тела передаются кадрами; chunked-кодирование HTTP/1.1 для этого не используется.
- В HTTPS согласование HTTP/2 обычно происходит через ALPN с идентификатором h2.

Границы преимущества HTTP/2

- Мультиплексирование уменьшает блокировку между ответами на уровне HTTP.
- Потери пакетов в TCP могут задерживать доставку данных всех потоков соединения. HTTP/2 не устраняет эту блокировку TCP.
- Приоритизация выражает предпочтения; она не гарантирует порядок или время завершения ответов.
- Реальная скорость зависит также от сети, размеров ресурсов и работы сервера.

Методы: смысл запроса

- GET получает представление ресурса; не предназначен для изменения состояния.
- POST просит ресурс обработать данные по своей семантике; создание ресурса - частый случай.
- PUT задаёт замену состояния целевого ресурса; DELETE запрашивает удаление его связи с текущей функциональностью.
- PATCH применяет частичные изменения; HEAD получает метаданные как для GET, без тела ответа.
- Метод не заменяет проверку прав доступа и правил приложения.

Классы ответов

- 1xx: промежуточная информация о выполнении обмена.
- 2xx: запрос успешно обработан; пример 200 OK.
- 3xx: дополнительное действие клиента, часто перенаправление. 304 используется для проверки кэша.
- 4xx: запрос не может быть выполнен по причине на стороне запроса; примеры 400, 404, 405.
- 5xx: сервер не смог выполнить допустимый запрос; пример 500.

Маршрутизация в Go

Фрагмент main; импорты net/http, time, log; обработчики на следующих слайдах

```
mux := http.NewServeMux()
mux.HandleFunc("/", home)
mux.HandleFunc("/hello", hello)
mux.HandleFunc("/api/data", data)

server := &http.Server{
    Addr:              "127.0.0.1:8080",
    Handler:           mux,
    ReadHeaderTimeout: 5 * time.Second,
}
log.Fatal(server.ListenAndServe())
```

- ServeMux выбирает обработчик, а http.Server обслуживает соединения.
- Адрес ограничен локальным компьютером для учебной практики.

Корневой обработчик

Функция; импорты fmt и net/http

```
func home(w http.ResponseWriter, r *http.Request) {
    if r.URL.Path != "/" {
        http.NotFound(w, r)
        return
    }
    if r.Method != http.MethodGet {
        w.Header().Set("Allow", http.MethodGet)
        http.Error(w, "method not allowed", 405)
        return
    }
    w.Header().Set("Content-Type", "text/html; charset=utf-8")
    _, _ = fmt.Fprint(w, "<h1>Hello, Go!</h1>")
}
```

Заголовки задаются до записи тела. Запись тела без явного WriteHeader отправляет статус 200.

Query-параметр и текстовый ответ

Функция; импорты fmt и net/http

```
func hello(w http.ResponseWriter, r *http.Request) {
    if r.Method != http.MethodGet {
        w.Header().Set("Allow", http.MethodGet)
        http.Error(w, "method not allowed", 405)
        return
    }
    name := r.URL.Query().Get("name")
    if name == "" { name = "guest" }
    w.Header().Set("Content-Type", "text/plain; charset=utf-8")
    _, _ = fmt.Fprintf(w, "Hello, %s!", name)
}
```

- В /hello?name=Go параметр name имеет значение Go.
- Ответ text/plain не интерпретирует пользовательский ввод как HTML. Для HTML-шаблонов используйте html/template.

JSON-ответ

Функция; импорты encoding/json, net/http и log

```
func data(w http.ResponseWriter, r *http.Request) {
    if r.Method != http.MethodGet {
        w.Header().Set("Allow", http.MethodGet)
        http.Error(w, "method not allowed", 405)
        return
    }
    payload := map[string]string{"key": "value"}
    w.Header().Set("Content-Type", "application/json")
    if err := json.NewEncoder(w).Encode(payload); err != nil {
        log.Printf("write JSON: %v", err)
    }
}
```

Ошибка записи может возникнуть после отправки заголовков. Повторный HTTP-ответ в этот момент уже не исправит статус.

Сборка и проверка локального сервера

Команды; на Linux/macOS используйте curl вместо curl.exe

```
go run main.go

# В другом терминале Windows:
curl.exe -i http://127.0.0.1:8080/
curl.exe -i "http://127.0.0.1:8080/hello?name=Go"
curl.exe -i http://127.0.0.1:8080/api/data
curl.exe -i http://127.0.0.1:8080/missing
curl.exe -i -X POST http://127.0.0.1:8080/hello
```

- Ожидаем HTML 200, текст 200, JSON 200, отсутствующий путь 404 и запрещённый метод 405.
- Остановить учебный процесс: Ctrl+C.

HTTP и HTTPS в учебном примере

- Обычный ListenAndServe здесь показывает HTTP на localhost, а не демонстрирует автоматически HTTP/2.
- Для HTTPS нужен TLS; поддержку HTTP/2 проверяют отдельно с учётом конфигурации сервера и клиента.
- Стандартные порты: HTTP - 80, HTTPS - 443. Для локальной практики выбран 8080.
- Обычный некорректный запрос не означает остановку всего сервера. Сервер продолжает принимать соединения, пока не остановлен или не возникла ошибка обслуживания сервера.

Чему научились

- Различать семантику HTTP и способ передачи сообщений.
- Объяснять ограничения pipelining и пользу мультиплексирования.
- Не путать HTTP-блокировку с потерями на уровне TCP.
- Создавать ServeMux, читать query-параметры и писать текст/JSON.
- Проверять статус, Content-Type, тело и обработку ошибочных запросов.

Материалы и источники

Учебная реконструкция по сохранившемуся конспекту урока 9.1. Уточнены постоянные соединения HTTP/1.1, пределы мультиплексирования и поведение сервера. Пример локального сервера не выдаётся за демонстрацию HTTP/2. Утраченный оригинальный PDF не найден.

https://www.rfc-editor.org/rfc/rfc9110
https://www.rfc-editor.org/rfc/rfc9112
https://www.rfc-editor.org/rfc/rfc9113
https://pkg.go.dev/net/http